Failure modes

Three ways a **correctly functioning** version of this tool produces a bad outcome. None are bugs.

> *Fixing the bugs makes all three worse, because a tool that is often wrong gets checked, and a tool that is reliably right stops being checked at exactly the point where it is silently wrong.*

1. The image model draws a claim no text gate can read

WHAT HAPPENS

Nano Banana Pro is conditioned on the real product photograph, so the pack comes back looking right. Looking right is the problem.

The model redraws the bottle rather than compositing it, and on this brand the number printed on the front of the pack *is* the regulated claim. A 10% Niacinamide pack rendered with a 12% or 20% label is a false claim about a real product, published at media spend.

It passes everything I built. The claim gate in `src/lib/policy/check.ts` reads the copy layer:

– headline

– subhead

– body

– the text string handed to the image model

The digits the model paints onto the bottle are not in any of those fields. They are pixels, and no gate in this codebase reads pixels.

The risk is worst exactly where this brand's creatives live. Its ads are text-heavy by design: leader lines to small-caps labels, callout boxes, a concentration set in large type. Every one of those is a surface for the model to hallucinate a digit, and the more type in the frame the more chances it gets.

A one-digit error is also the version that actually ships, because a creative that looks correct is the one a reviewer approves fastest.

WHY IT ISN'T A BUG

The model is doing what it was asked. There is no defect to fix, only a check that does not exist.

FIX

A **pixel-level claim gate**: OCR the rendered creative, pull every number and unit off the depicted packaging and off any large type in the frame, and run the same allowlist already applied to copy. A mismatch blocks identically to a bad headline.

If OCR proves unreliable at small type, the fallback is structural rather than clever: composite the real photograph and let the model generate only the environment around it.

BEFORE LAUNCH

This gates going live at all. Until it exists, run with the pack held below legible size, or treat generated creatives as internal comps that a designer rebuilds before spend.

> *I did not build this check and I would not put media budget behind the tool without it.*

2. The tool's picture of Minimalist is frozen, and nothing in it notices when the brand moves

Three things were derived once, from one reading of the brand at one date:

1. the voice and compliance rules
2. the visual identity
3. the shape of the storefront JSON

All three go stale, in different ways, and none of them announces it.

WHAT HAPPENS: THE RULES

The brand launches a product at a concentration the rules do not know, or runs a festive campaign in a register the voice rules call off-brand, or adopts a new colour into the palette. The scorer keeps returning 84 out of 100 with a confident rationale. A number with a rationale reads as an institutional standard rather than as one person's judgment from a date.

The damage is not the wrong score. It is that the scorer starts blocking legitimate new work, the team learns to override it, overriding becomes the habit, and the habit carries into the compliance findings where it was right. The tool loses authority precisely where it had value, and loses it quietly.

WHAT HAPPENS: THE STOREFRONT

The claim gate's entire allowlist is scraped, so the gate is only ever as good as the parse behind it. Two places are brittle by construction:

– **Concentrations** are pulled out of the *product title* with a regex (`extractConcentrations` in `src/lib/scrape/shopify.ts`), so a title format change empties or corrupts the allowlist.

– **Price** is read from two Shopify endpoints that report different units, `/products.json` in decimal rupees and `/products/<handle>.js` in integer paise, normalised in exactly one place.

A Shopify migration, a theme change, a headless replatform, or a key rename breaks that quietly.

The failure is asymmetric, and the bad half is the quiet one.

PARSE RETURNS	WHAT HAPPENS
Nothing	Every numeric claim blocks. Somebody notices within an hour.
The wrong number	The gate authorises it and prints it with full confidence. The tool built to prevent false claims has just manufactured one.

WHY IT ISN'T A BUG

Nothing is broken today. The brand moving is the normal case, not the exception.

FIX

Three, in cost order.

1. **Version and date the ruleset**, and print the version on every score, so the standard is legibly a document with an author.
2. **Make disagreement a first-class input**: an override on any finding that requires a written reason, captured. Overrides are the only signal that will ever tell you which rules are wrong, and they are free to collect.
3. **Add a parse canary**: a scheduled read of a handful of known products asserting the shape and the values, failing loudly on drift rather than at the next creative.

BEFORE LAUNCH

The version stamp and the parse canary. Both are cheap and both change how every downstream output is read.

Override capture and a standing rule review follow once there is enough traffic for the overrides to say something.

---

3. The scorer sounds equally confident whether it verified anything or not, and it does not repeat itself

Two mechanisms, one presentation. Both end with a reviewer trusting a number that has not earned it.

WHAT HAPPENS: IT CANNOT VERIFY WHAT IT WAS NOT GIVEN

The product URL is optional. Without one, product-specific claims come back marked `verified: false` rather than silently passing, which is the right design and is where the honesty stops.

> *A disclaimer sitting beside a confident 78 out of 100 is not a guardrail, it is a footnote, and footnotes do not change behaviour.*

A creative stating the wrong concentration for a real product scores well, reads as approved, and the one line saying the claim was not checked is the line nobody reads.

WHAT HAPPENS: IT DOES NOT REPEAT ITSELF

The scorer already runs at `temperature: 0` (`src/lib/pipeline/score.ts`). That is worth stating plainly, because the obvious lever is already pulled: **temperature 0 is not determinism.** Batching, floating-point non-associativity and any model version change all move the output, so the same creative scored twice gives different findings and a different number.

The range being narrow does not make this survivable, and this is where I would push back on treating it as a cosmetic inconsistency. **The verdict boundary sits *inside* the range.** `aggregate()` marks a creative `fix_required` when `overall < 70`, so 68 and 71 are the same creative with opposite outcomes.

Once a marketer notices that re-running sometimes flips a fail to a pass, re-running is free and takes eight seconds. The scorer stops being a gate and becomes a dice roll with a retry button, and the compliance findings it was actually good at get re-rolled along with everything else.

WHY IT ISN'T A BUG

Both behaviours are the system working as specified. The first is the honest handling of missing evidence; the second is what generative scoring is.

FIX

Build a **human-scored eval set**: fifty to a hundred real creatives, passed and failed, scored by the people whose judgment the tool is meant to encode. That set does three jobs a prompt cannot:

1. It calibrates the absolute numbers against something other than the model's self-consistency.
2. It measures run-to-run variance, so the threshold can be set outside the noise band, or replaced with a band and a "borderline, send to a human" verdict.
3. It becomes the regression test for every prompt and model change, which is the only way to know an upgrade did not quietly move the standard.

Alongside it, two smaller changes:

– Make unverified a **verdict state** rather than a footnote, so a creative with unverifiable product claims cannot return `pass` at all.

– Show the run count on any creative scored more than once, so score shopping is visible in the record rather than invisible in it.

BEFORE LAUNCH

The unverified verdict state, and pinning the model version. Both are small and both stop the worst reading of the output.

The eval set is the real answer and it is a week of somebody's time, so it lands after launch, but nothing else in this document should be trusted until it exists.

Summary

#	FAILURE	COST IF IT LANDS	BEFORE LAUNCH	AFTER
1	Fabricated concentration on the pack	False regulated claim at media spend	Pixel-level claim gate, or keep the pack illegible	Composite real photography
2	Frozen rules, palette and storefront parse	Wrong claims authorised; tool overridden into irrelevance	Ruleset version stamp, parse canary	Override capture, standing rule review
3	Uncalibrated, non-repeating scores	Score shopping; unverified claims read as approved	Unverified as a verdict, pinned model version	Human-scored eval set

THE ONE I AM LEAST ABLE TO DEFEND

Failure mode 1. Everything else here is something I would improve. That one is something I would gate the launch on, and I did not build the check.